

LAPORAN RESMI
“Mengelola Kelas”



Disusun oleh:

Nama: Kristian Utama Putra

NRP: 3125600047

Kelas: 2 STr. Teknik Informatika B

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

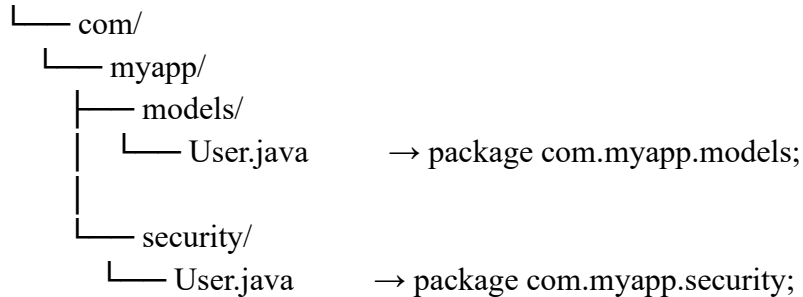
2026

Tugas Pendahuluan

1. Apakah yang dimaksud dengan package?

Package adalah cara untuk manage class-class yang ada, terutama jika jumlah classnya sangat banyak. Keuntungannya berupa:

- Kejelasan direktori: lebih mudah dibaca/kelola
- Fleksibilitas: 2 class dengan nama yang sama masih bisa ada



2. Apakah kegunaan kata kunci import?

Agar kita bisa pakai class dari package lain tanpa menulis ulang addressnya dengan lengkap.

<pre>package com.myapp.controllers; public class UserController { public void process() { com.myapp.models.User u = new com.myapp.models.User(); // ribet & panjang } }</pre>	Ribet
<pre>package com.myapp.controllers; import com.myapp.models.User; // import class User dari package models public class UserController { public void process() { User u = new User(); // cukup nama class-nya aja } }</pre>	Simpel

3. Apakah kegunaan kata kunci this?

- untuk menunjukkan suatu member dalam classnya sendiri. This dapat digunakan baik untuk data member maupun untuk function member, serta dapat juga digunakan untuk konstruktor.

```
public class User {
    private String name;
    private String email;
```

Ini field

Ini parameter

```
// constructor  
public User(String name, String email) {  
    this.name = name;  
    this.email = email;  
}  
}
```

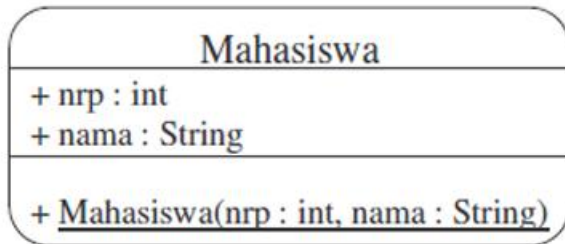
this.data_member → merujuk pada data member

this.function_member() → merujuk pada function member

this() → merujuk pada konstruktor

Percobaan

Percobaan 1 : Menggunakan kata kunci this

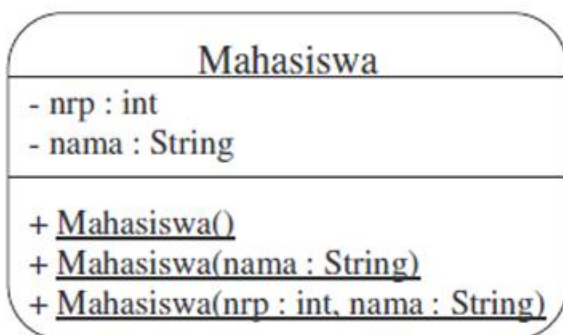


Dari class diagram tersebut, dapat diimplementasikan ke dalam program sebagai berikut:

```
public class Mahasiswa {
    public int nrp;
    public String nama;

    public Mahasiswa(int nrp, String nama) {
        this.nrp=nrp;
        this.nama=nama;
    }
}
```

Percobaan 2 : Memakai kata kunci this pada overloading constructor



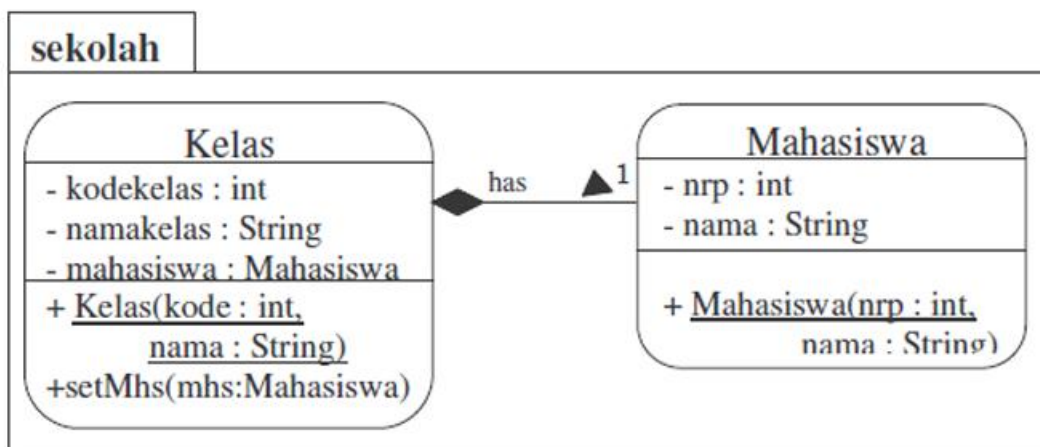
Dari class diagram tersebut, dapat diimplementasikan ke dalam program sebagai berikut:

```

public class Mahasiswa {
    private int nrp;
    private String nama;
    public Mahasiswa() {
        this(0, "");
    }
    public Mahasiswa(String nama) {
        this(0, nama);
    }
    public Mahasiswa(int nrp, String nama) {
        this.nrp = nrp;
        this.nama = nama;
    }
}

```

Percobaan 3 : Menggunakan package dan import



Dari class diagram tersebut, dapat diimplementasikan ke dalam program dibawah ini.

Sebelum melakukan kompilasi, daftarkan direktori tempat package diatas disimpan.

```

package sekolah;

public class Kelas {
    private int kodekelas;
    private String namakelas;
    private Mahasiswa mahasiswa;

    public Kelas(int kode, String nama) {
        this.kodekelas = kode;
        this.namakelas = nama;
    }

    public void setMhs(Mahasiswa mhs) {
        this.mahasiswa = mhs;
    }
}

package sekolah;

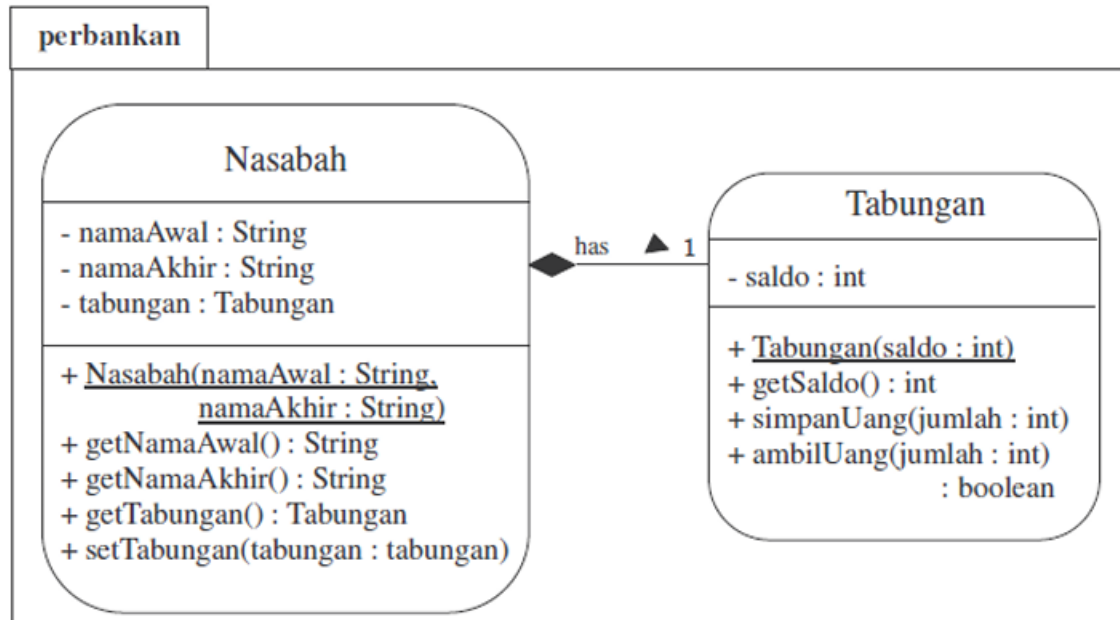
public class Mahasiswa {
    private int nrp;
    private String nama;

    public Mahasiswa(int nrp, String nama) {
        this.nrp = nrp;
        this.nama = nama;
    }
}

```

Latihan

Latihan 1: Mengimplementasikan UML class diagram dalam program untuk package perbankan



SC:

```
package w7.perbankan;

public class Nasabah {
    private String namaAwal;
    private String namaAkhir;
    private Tabungan tabungan;

    public Nasabah(String namaAwal, String namaAkhir){
        this.namaAwal = namaAwal;
        this.namaAkhir = namaAkhir;
    }

    public String getNamaAwal(){
        return namaAwal;
    }

    public String getNamaAkhir(){
        return namaAkhir;
    }

    public Tabungan getTabungan(){
        return tabungan;
    }

    public void setTabungan(Tabungan tabungan){
        this.tabungan = tabungan;
    }
}
```

```
package W7.perbankan;

public class Tabungan {
    private int saldo;

    public Tabungan(int saldo){
        this.saldo = saldo;
    }

    public int getSaldo(){
        return saldo;
    }

    public void simpanUang(int jumlah){
        this.saldo += jumlah;
    }

    public boolean ambilUang(int jumlah){
        if (jumlah <= saldo) {
            this.saldo -= jumlah;
            return true;
        }
        return false;
    }
}
```

```

import W7.perbankan.*;

public class TesLatihan {
    public static void main(String[] args) {
        int tmp;
        boolean status;

        Nasabah nasabah = new Nasabah("Agus","Daryanto");
        System.out.println("Nasabah atas nama : "+nasabah.getNamaAwal() + " " +
nasabah.getNamaAkhir());

        nasabah.setTabungan(new Tabungan (5000));
        tmp = nasabah.getTabungan().getSaldo();
        System.out.println("Saldo awal : "+tmp);
        nasabah.getTabungan().simpanUang(3000);
        System.out.println("Jumlah uang yang disimpan 3000");
        status=nasabah.getTabungan().ambilUang(6000);
        System.out.print("Jumlah uang yang diambil 6000");
        if(status)
            System.out.println(" OK");
        else
            System.out.println(" Gagal");

        nasabah.getTabungan().simpanUang(3500);
        System.out.println("Jumlah uang yang disimpan 3500");
        status=nasabah.getTabungan().ambilUang(4000);
        System.out.print("Jumlah uang yang diambil 4000");
        if(status)
            System.out.println(" OK");
        else
            System.out.println(" Gagal");

        status=nasabah.getTabungan().ambilUang(1600);
        System.out.print("Jumlah uang yang diambil 1600");
        if(status)
            System.out.println(" OK");
        else
            System.out.println(" Gagal");

        nasabah.getTabungan().simpanUang(2000);
        System.out.println("Jumlah uang yang disimpan 2000");
        tmp=nasabah.getTabungan().getSaldo();
        System.out.println("Saldo sekarang = "+tmp);

    }
}

```

Project Structure:

OOP\

├── TesLatihan.java

└── W7\

└── perbankan\

└── Nasabah.java

└── Tabungan.java

Output:

```
Nasabah atas nama : Agus Daryanto
Saldo awal : 5000
Jumlah uang yang disimpan 3000
Jumlah uang yang diambil 6000 OK
Jumlah uang yang disimpan 3500
Jumlah uang yang diambil 4000 OK
Jumlah uang yang diambil 1600 Gagal
Jumlah uang yang disimpan 2000
Saldo sekarang = 3500
```

TUGAS

Tugas 1 : Perhatikan program dibawah ini. Apa yang terjadi bila dikompile dan dijalankan? Jelaskan jawaban anda!

```
public class Pegawai{
    int nip;
    String nama;
    public Pegawai(int nip_pegawai){
        this(nip_pegawai,"NoName");
    }

    public Pegawai(int nip_pegawai, String nama_pegawai){
        this.nip = nip_pegawai;
        this.nama = nama_pegawai;
    }
}
```

Output:

```
Error: Main method not found in class Pegawai, please define the main method as:
    public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```

Analisis:

Kode berhasil dicompile sih tapi karena tidak ada main methodnya jadi muncul error, coba kita tambahkan main method:

```
public static void main(String[] args){
    Pegawai p1 = new Pegawai(101);
    Pegawai p2 = new Pegawai(102, "Budi");
    System.out.println("NIP: " + p1.nip + ", Nama: " + p1.nama);
    System.out.println("NIP: " + p2.nip + ", Nama: " + p2.nama);
}
```

Output:

```
NIP: 101, Nama: NoName
NIP: 102, Nama: Budi
```

Analisis:

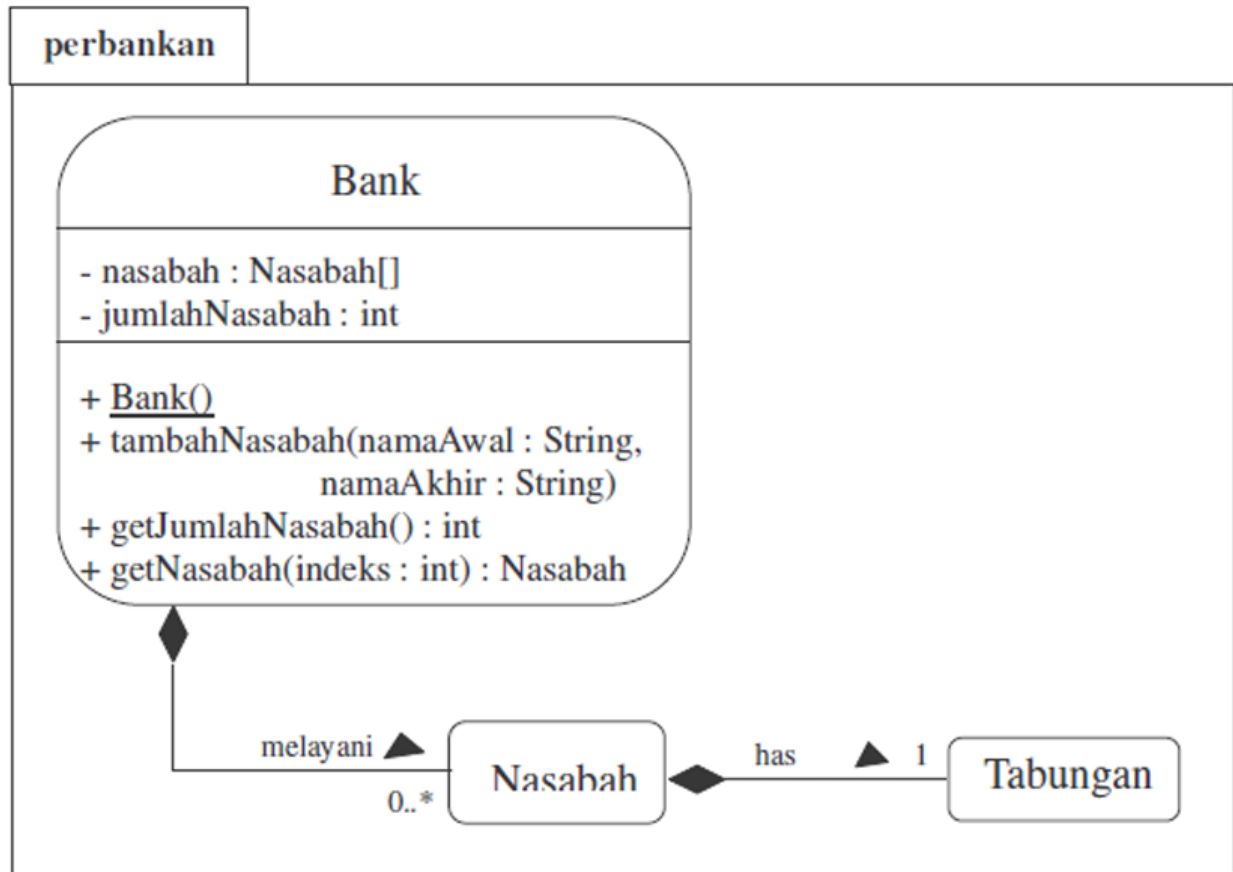
Blueprint class pegawai di atas biasanya disebut juga sebagai “overloading constructor” yang berarti dia punya satu nama method/constructor yang sama, tapi ada beberapa versi dengan parameter berbeda. Dalam case ini:

Sama-sama bernama Pegawai (nama constructor wajib sama dengan nama class), tapi jumlah parameternya beda:

- Pegawai(int nip_pegawai) → 1 parameter
- Pegawai(int nip_pegawai, String nama_pegawai) → 2 parameter

Ketika Cuma input NIP maka nama otomatis diset “NoName” tapi ketika input NIP dan Nama maka langsung dipakai.

Tugas 2. Mengembangkan package perbankan dengan tambahan class Bank



SC:

```
import w7.perbankan.*;

public class TesTugas {
    public static void main(String arg []){
        Bank bank = new Bank();
        bank.tambahNasabah("Agus", "Daryanto");
        bank.getNasabah(0).setTabungan(new Tabungan(5000));
        bank.tambahNasabah("Tuti", "Irawan");
        bank.getNasabah(1).setTabungan(new Tabungan(7000));
        bank.tambahNasabah("Ani", "Ratna");
        bank.getNasabah(2).setTabungan(new Tabungan(4000));
        bank.tambahNasabah("Bambang", "Darmawan");
        bank.getNasabah(3).setTabungan(new Tabungan(6500) );
        System.out.println("Jumlah Nasabah = "+bank.getJumlahNasabah());
        for(int i =0; i<bank.getJumlahNasabah(); i++){
            System.out.println("Nasabah ke-" + (i+1) + " : " +
                bank.getNasabah(i).getNamaAwal() + " " +
                bank.getNasabah(i).getNamaAkhir() + " ; Saldo = " +
                bank.getNasabah(i).getTabungan().getSaldo());
        }
    }
}
```

Output yang diharapkan:

```
Jumlah nasabah = 4
Nasabah ke-1 : Agus Daryanto ; Saldo = 5000
Nasabah ke-2 : Tuti Irawan ; Saldo = 7000
Nasabah ke-3 : Ani Ratna ; Saldo = 4000
Nasabah ke-4 : Bambang Darmawan ; Saldo = 6500
```

Output real:

```
Jumlah Nasabah = 4
Nasabah ke-1 : Agus Daryanto ; Saldo = 5000
Nasabah ke-2 : Tuti Irawan ; Saldo = 7000
Nasabah ke-3 : Ani Ratna ; Saldo = 4000
Nasabah ke-4 : Bambang Darmawan ; Saldo = 6500
```

Analisis:

Intinya, file TesTugas.java ini untuk ngetes apakah ekosistem perbankan kita berjalan dengan benar. Kita ngetes class Nasabah dan Tabungan sudah bekerja dengan benar atau belum dan ternyata sudah.